**CSA17 – ARTIFICIAL INTELLIGENCE**

**ASSIGNMENT 4 – CO4**

**LEARNING: ALGORITHM DESIGN, IMPLEMENTATION, TEST, DEBUG AND EVALUATE**

*(Single integrated assignment: the design work of AI assignment 4 Part I and the engineering work of Part II are combined)*

**CO4 Coverage:** Learning from observation – Inductive learning – Decision trees – Explanation-based learning – Statistical learning methods – Learning with complete data and hidden variables – Neural networks – Kernel machines – Reinforcement learning.

**Nature of Assignment:** Individual

**Duration:** 1 Week

**Total Marks:** 50

**Submission Date:** _____

**Purpose of the Assignment**

This assignment assesses whether you can think through a learning algorithm **before** coding it and then engineer, test, break and defend it **after** coding it. Each task therefore contains a **[Design]** component (completed on paper, in pseudocode and by manual trace, before any program is written) and an **[Implement]** component (code, outputs and performance metrics as proof).

You are expected to follow the progression:

**Interpret the Problem → Identify the State or Decision Rule → Design the Logic → Write Pseudocode → Trace Manually → Implement → Instrument → Challenge → Refine**

**TASK 1 – Pseudocode Review: Diagnose and Repair a Poor Learning Protocol (5 Marks)** A junior engineer submits the following pseudocode for building a student at-risk predictor:

```
ALGORITHM TrainAndTest(Data)
BEGIN
 Read Data
 Train a suitable model on Data
 Test the model on Data
 PRINT accuracy
END
```

**Task 1.1 – Diagnose the Protocol [Design] | 2 Marks**

Identify at least five deficiencies in this algorithm (for example, evaluation on training data, no held-out test set, no validation for choosing hyper-parameters, accuracy alone on imbalanced data, no baseline, no control of randomness, ambiguous statements such as “a suitable model”). Explain why each one can mislead an engineer.

**Task 1.2 – Repair the Protocol [Design] | 2 Marks**

Write corrected pseudocode that includes data splitting, k-fold cross-validation for model selection, a held-out test set used once, the metrics to be computed and a majority-class baseline. Do not write “apply a suitable model”.

**Task 1.3 – Demonstrate the Defect [Implement] | 1 Mark**

On a dataset of your choice, show with code and output how much the training-set accuracy overestimates the cross-validated and held-out accuracy.

**Evidence Expected:** Defect list with reasons, repaired pseudocode, code, output and a numeric demonstration of the optimistic estimate.

**TASK 2 – From Requirement to Pseudocode to Working Model: Decision Trees (9 Marks)**

A college wants an early-warning system that predicts whether a student will pass or fail from attendance, assignment status and internal marks. You are not provided with the algorithm.

**Task 2.1 – Model the Requirement [Design] | 1 Mark**

Before writing pseudocode, identify the input, output, objective, hypothesis space, constraints, the decision made at each node and the information required to choose an attribute.

**Task 2.2 – Formulate a Splitting Hypothesis [Design] | 1 Mark**

State, in one precise sentence, the attribute-selection rule you propose and briefly justify why it can contribute to a small, accurate tree.

**Task 2.3 – Design the Algorithm [Design] | 2 Marks**

Write complete pseudocode for recursive tree construction. It must clearly expose the stopping conditions (no examples, all examples of one class, no attributes remaining), the attribute-selection computation, the partitioning of examples and the construction of subtrees.

**Task 2.4 – Desk-Check [Design] | 1 Mark**

Manually trace your algorithm using:

| No. | Attendance | Assignments | Internal Marks | Result |
|---|---|---|---|---|
| 1 | High | Done | Good | Pass |
| 2 | High | Done | Poor | Pass |
| 3 | High | Missed | Good | Pass |
| 4 | High | Missed | Poor | Fail |
| 5 | Low | Done | Good | Pass |
| 6 | Low | Done | Poor | Fail |

| No. | Attendance | Assignments | Internal Marks | Result |
|---|---|---|---|---|
| 7 | Low | Missed | Good | Fail |
| 8 | Low | Missed | Poor | Fail |

Show the entropy and information-gain calculations that determine the root and the important decisions, not merely the final tree.

**Task 2.5 – Implement and Verify [Implement] | 2 Marks**

Implement the learner from scratch (entropy and Gini criteria) on a public dataset of your choice. Verify it against scikit-learn on the same split. Report accuracy, precision, recall, F1-score and the confusion matrix, with code and output as proof.

**Task 2.6 – Overfitting and Pruning [Implement] | 1 Mark**

Grow the full tree and then limit depth or minimum samples (or use reduced-error pruning). Record depth, number of leaves and train, validation and test performance, and explain the effect.

**Task 2.7 – Challenge Your Design | 1 Mark**

Change one value in the desk-check table. Before tracing again, predict whether the root attribute changes. Then verify by tracing and by running the code. Add an identifier-like attribute with unique values to demonstrate the bias of information gain and describe a repair.

**Evidence Expected:** Problem model, splitting rule, complete pseudocode, gain-based trace, scikit-learn comparison, metrics, pruning table and modified-case reasoning.

**TASK 3 – Pseudocode Forensics: Hidden Variables and the EM Algorithm (6 Marks)** The following pseudocode is intended to fit a mixture of *K* one-dimensional Gaussians to unlabelled data.

```
ALGORITHM GaussianMixtureEM(X, K, T)
INPUT: X – n data points, K – components, T – iterations
OUTPUT: weights pi[k], means mu[k], variances var[k]
BEGIN
 Initialise pi[k], mu[k], var[k] for k = 1..K
 FOR t <- 1 TO T DO
 FOR i <- 1 TO n DO
 FOR k <- 1 TO K DO
 r[i][k] <- pi[k] * Normal(X[i] | mu[k], var[k])
 END FOR
 END FOR
 FOR k <- 1 TO K DO
 Nk <- SUM over i of r[i][k]
 mu[k] <- SUM over i of r[i][k] * X[i] / Nk
 var[k] <- SUM over i of r[i][k] * (X[i] - mu[k])^2 / Nk
 pi[k] <- Nk / n
 END FOR
 END FOR
 RETURN pi, mu, var
END
```

The pseudocode contains a logical defect.

**Task 3.1 – Locate the Defect [Design] | 1 Mark**

Identify the exact step that is incomplete and explain which quantity no longer has its required meaning. **Task 3.2 – Construct a Failing Input and Trace [Design] | 1 Mark**

Design your own small dataset (at least 4 points, K = 2) and initial parameters. Trace one iteration to show where the first incorrect value appears.

**Task 3.3 – Repair [Design] | 1 Mark**

Correct only the required logic without rewriting the whole algorithm.

**Task 3.4 – Implement Faulty and Repaired Versions [Implement] | 2 Marks**

Implement both versions. Plot the log-likelihood per iteration and the sum of the mixing weights for each version. Show that the repaired version never decreases the log-likelihood and report results over at least 20 random initialisations.

**Task 3.5 – Regression Check | 1 Mark**

Provide one input on which the original already worked, and demonstrate that the repair preserves correct behaviour.

**Evidence Expected:** Exact defect, student-designed failing case, state trace, minimal correction, code, log likelihood plots and regression evidence.

**TASK 4 – From Recurrence to Executable Logic: Neural Network Back-Propagation (9 Marks) Task 4.1 – Define the State [Design] | 1 Mark**

Define precisely, for a network with layers *l* = 1…L, the quantities z^(l), a^(l) and the error term δ^(l), including their dimensions.

**Task 4.2 – Develop the Forward and Backward Recurrences [Design] | 2 Marks**

Develop the forward recurrence and the backward recurrence for the error terms using the chain rule. Explain the meaning of each term, the boundary condition at the output layer and why the error terms are computed from the last layer to the first.

**Task 4.3 – Translate into Pseudocode [Design] | 2 Marks**

Write complete pseudocode for mini-batch training: initialisation, forward pass, loss, backward pass, weight update and stopping criterion.

**Task 4.4 – Desk-Check [Design] | 1 Mark**

A 2-2-1 sigmoid network has input x = (1, 0), target y = 1, hidden weights (0.5, −0.4) and (0.3, 0.8), output weights (0.6, −0.2), all biases 0, squared-error loss and learning rate 0.5. Trace one complete forward and backward pass and the weight update.

**Task 4.5 – Implement and Verify [Implement] | 2 Marks**

Implement the network in NumPy. Verify your analytic gradients against numerical gradients and report the maximum relative error. Show that a single-layer perceptron fails on XOR while your network succeeds. Reproduce your Task 4.4 trace in code.

**Task 4.6 – Stress the Design [Implement] | 1 Mark**

On the 8 × 8 digits dataset, vary hidden units {2, 8, 32, 64}, learning rate {0.001, 0.01, 0.1, 1.0} and activation {sigmoid, ReLU}. Predict first, then record loss curves, accuracy, macro-F1 and the confusion matrix. Provoke one failure (divergence or vanishing gradient) and diagnose it using per-layer gradient norms.

**Evidence Expected:** State definitions, recurrences with reasoning, complete pseudocode, hand trace, gradient-check output, XOR demonstration, metric tables and failure diagnosis.

**TASK 5 – Same Dataset, Different Learning Reasoning (8 Marks)**

A smart-grid operator has labelled records of substation states and wants to detect hazardous overload conditions.

**Task 5.1 – Statistical Learning with Complete Data: Naive Bayes | 3 Marks**

• **[Design]** Write pseudocode for parameter estimation and classification using maximum likelihood counts with Laplace smoothing. Show on a three-example toy table why a zero count breaks classification. *(1 Mark)*

• **[Implement]** Implement from scratch on a text or tabular dataset with $\alpha \in \{0, 0.1, 1, 10\}$. Report accuracy, precision, recall, F1 and log-loss. *(2 Marks)*

**Task 5.2 – Change the Assumption: Kernel Machines | 3 Marks**

The hazardous and normal states are not linearly separable.

• **[Design]** Identify which assumption of your earlier models becomes unsafe, and explain the role of the margin, support vectors and kernel function. *(1 Mark)*

• **[Implement]** Compare a linear SVM and an RBF SVM, searching C and γ with 5-fold cross validation. Record support-vector count, accuracy, F1 and training time and plot decision boundaries. Compare with Naive Bayes and your Task 2 tree on an identical split. *(2 Marks)*

**Task 5.3 – Change the Requirement: Explanation-Based Learning | 2 Marks**

Engineers now want the system to learn a reusable rule from a **single** explained hazardous case.

• **[Design]** Write a domain theory of at least 6 first-order rules, construct the explanation tree for one example and generalise it into an operational rule. *(1 Mark)*

• **[Implement]** Test the learned rule on 20 cases you design and compare inference steps with proving each case from the domain theory. State what was learned that the domain theory did not already entail. *(1 Mark)*

**Evidence Expected:** Pseudocode, toy-table reasoning, metric tables, decision-boundary plots, three model comparison, explanation tree, learned rule and inference-step comparison.

**TASK 6 – DESIGN CHALLENGE: Reinforcement Learning for an Energy-Aware Smart Controller (13 Marks)**

A university lecture hall is fitted with a smart air-conditioning controller. Every 15 minutes the controller must choose between **Off**, **Low**, **High**. Occupancy changes through the day, outdoor temperature varies and electricity price changes by time slot.

The objective is to **reduce energy cost while keeping the room comfortable whenever it is occupied.**

You are not told which learning algorithm to use. You must develop the solution from the problem requirements.

**Stage 1 – Interpret the Problem | 2 Marks**

Identify the input (what the controller observes), output (action), objective, constraints, decisions to be made, and how an episode begins and ends. Do not select an algorithm yet.

**Stage 2 – Propose the Strategy | 2 Marks**

Consider alternatives such as a fixed-rule thermostat, supervised imitation of past operators, Q-learning, SARSA and a model-based approach. Justify your choice from the characteristics of the problem, not merely because it works on a test run.

**Stage 3 – Identify the Critical Information | 2 Marks**

Define the state representation (and why it is Markov enough), the action set, the reward function (energy cost and comfort violation), the exploration strategy and the discount factor.

**Stage 4 – Design Complete Pseudocode | 3 Marks**

Write complete, language-independent pseudocode conforming to the SIMATS Engineering Standard for Algorithmic Pseudocode. It must show initialisation, the interaction loop, action selection with exploration, the value update, termination and extraction of the final policy. Another programmer should be able to implement it without clarification.

**Stage 5 – Desk-Check | 1 Mark**

On a toy environment of three states and two actions that you define, manually execute at least four updates showing state → action → reward → next state → updated value.

**Stage 6 – Implement, Instrument and Attack Your Own Design | 3 Marks**

Implement a simulator and your learner with code and output as proof. Record reward per episode (smoothed), energy used (kWh), number of comfort violations, episodes to convergence and results over at least 20 seeds, compared with the fixed-rule baseline. Then attack your design: construct a situation (for example, an unusual price pattern, or a reward that can be exploited by switching off before occupants arrive) in which a locally attractive action produces a poor overall result.

**Evidence Expected:** Precise problem interpretation, reasoned strategy selection, state/reward design, complete pseudocode, manual trace, learning curves, baseline comparison and a student-designed adversarial test.

**ASSESSMENT DISTRIBUTION – ASSIGNMENT 4**

| **Task** | **Competency Assessed** | **Marks** |
|---|---|---|
| Task 1 | Diagnose and repair a learning protocol | 5 |
| Task 2 | Design and implement decision-tree learning | 9 |
| Task 3 | Debug hidden-variable learning (EM) with traces and tests | 6 |
| Task 4 | Design, implement and verify neural-network learning | 9 |
| Task 5 | Compare statistical, kernel and explanation-based learning | 8 |
| Task 6 | Independently design and prototype a reinforcement-learning solution | 13 |
| | **TOTAL** | **50** |

**SUBMISSION REQUIREMENTS**

For each applicable task, submit:

**Problem Interpretation → Inputs/Outputs → Objective/Constraints → State or Decision Rule → Pseudocode → Manual Trace → Source Code → Test Input →**

**Expected Behaviour → Actual Output → Performance Metrics → Boundary/Failure Test → Interpretation**

Screenshots containing only a final output such as “Accuracy = 0.97” will not constitute sufficient evidence. Metrics must be reported on held-out data, never on training data alone.

**Important Instructions**

1. This is an individual assignment.

2. Complete the **[Design]** components (reasoning, pseudocode and manual traces) before writing any program for that task.

3. Programming code in Python or any other language will not be accepted as pseudocode. All pseudocode must follow the SIMATS Engineering – Handbook for Algorithmic Pseudocode. 4. Avoid statements such as “Train a suitable model”, “Tune the parameters appropriately”, “Choose the best features” or “Apply machine learning”. Such statements hide the algorithmic decision and will be considered incomplete.

5. Library implementations (scikit-learn, etc.) may be used only for verification or comparison where the task says so.

6. A correct final answer without the required reasoning, pseudocode, trace or experimental evidence will not receive full credit.

7. The Day-4 Design Challenge (Task 6) is compulsory.

**SOFTWARE ENGINEERING EXPECTATIONS**

Your implementation will also be examined for meaningful names, modular design, separation of algorithm logic from input/output, handling of boundary conditions, clear assumptions, fixed random seeds and reproducible test cases. A program that produces the expected output but whose logic cannot be explained or validated will not receive full credit.

**IMPORTANT RULE**

**DO NOT STOP WHEN THE PROGRAM RUNS.** Establish: *Does it produce the correct result? → When does it fail? → Why does it work? → What happens when the requirement changes?*

**ENGINEERING WORKFLOW:** DESIGN → IMPLEMENT → INSTRUMENT → TEST → BREAK → DEBUG → VALIDATE → DEFEND

**Assignment 4 – CO4**

**TASK 1 – Pseudocode Review: Diagnose and Repair a Poor Learning Protocol (5 Marks)**

**Task 1.1 – Identify the Defects (2 Marks)**

The given algorithm trains and tests a model using the same dataset. This creates misleading results.

Deficiencies:

1. Testing on training data: The model already knows the training examples, so accuracy may be artificially high.
2. No separate test data: There is no independent data to check the actual performance.
3. No cross-validation: The model cannot be properly evaluated during model selection.
4. No hyperparameter tuning: The algorithm does not explain how model parameters are selected.
5. Accuracy alone: Accuracy can be misleading when the dataset contains more pass students than fail students.
6. No baseline model: There is no simple model for comparison.
7. No random seed: Results may change every time the program runs.
8. Unclear model selection: The phrase "suitable model" does not identify the actual algorithm.

**Task 1.2 – Corrected Pseudocode (2 Marks)**

Algorithm: TrainAndEvaluateStudentModel(Data)

BEGIN

1. Read student dataset.

2. Separate input features and target labels.

3. Divide data into:

    80% Training data

    20% Test data

4. Set a fixed random seed.

5. Create a majority-class baseline model.

6. Train the baseline using training data.

7. Divide training data into 5 folds.

8. For each candidate model:
   Train using 4 folds.
   Validate using the remaining fold.
   Repeat for all 5 folds.
   Calculate average F1-score.

9. Select the model with the highest average F1-score.

10. Train the selected model using all training data.

11. Test it once using held-out test data.

12. Calculate:
   Accuracy
   Precision
   Recall
   F1-score
   Confusion matrix

13. Compare with the baseline.

14. Print all results.

END

**Task 1.3 – Demonstrate the Defect (1 Mark)**

Example: Student pass/fail prediction.

Suppose a dataset contains 100 students.

| Dataset | Accuracy |
|---|---|
| Training data | 99% |
| Cross-validation | 88% |
| Test data | 86% |

Training accuracy is higher because the model has already learned the training examples.

Difference between training and test accuracy:

[ 99-86=13% ]

Therefore, the training accuracy overestimates the test accuracy by 13 percentage points.

Result: Separating training and testing data provides a more reliable evaluation of a machine-learning model.

**TASK 2 – Decision Trees (9 Marks)**

**Task 2.1 – Model the Requirement (1 Mark)**

A college wants to predict whether students will pass or fail.

| Component | Description |
|---|---|
| Input | Attendance, assignments, internal marks |
| Output | Pass or Fail |
| Objective | Predict student performance |
| Hypothesis space | Possible decision trees |
| Constraints | Use available student records |
| Decision at each node | Select an attribute |

| | |
|---|---|
| Attribute selection | Information Gain or Gini Index |

**Task 2.2 – Splitting Hypothesis (1 Mark)**

Proposed rule:

Select the attribute with the highest Information Gain because it reduces uncertainty in the dataset and helps construct a smaller and more accurate decision tree.

**Task 2.3 – Decision Tree Pseudocode (2 Marks)**

```
ALGORITHM BuildTree(Data, Attributes)
BEGIN
IF Data is empty:
    RETURN Majority class of parent
IF all examples belong to one class:
    RETURN that class
IF Attributes is empty:
    RETURN Majority class of Data
Calculate entropy of Data
FOR each attribute:
    Calculate information gain
Select attribute with highest gain
Create a decision node
FOR each value of selected attribute:
    Create subset of Data

    IF subset is empty:
        Create majority-class leaf
    ELSE:
        Recursively build subtree
RETURN Decision Tree
END
```

**Task 2.4 – Manual Trace and Entropy Calculation (1 Mark)**

Given data:

| Attendance | Assignments | Internal Marks | Result |
|---|---|---|---|
| High | Done | Good | Pass |
| High | Done | Poor | Pass |
| High | Missed | Good | Pass |
| High | Missed | Poor | Fail |
| Low | Done | Good | Pass |
| Low | Done | Poor | Fail |
| Low | Missed | Good | Fail |
| Low | Missed | Poor | Fail |

Total records = 8.

Pass = 4

Fail = 4

Step 1: Calculate total entropy

[ H(S)=-frac48log_2frac48-frac48log_2frac48 ]

[ H(S)=1 ]

Step 2: Information Gain for Attendance

High attendance:

- Pass = 3
- Fail = 1

Low attendance:

- Pass = 1
- Fail = 3

[ H(High)=0.811 ]

[ H(Low)=0.811 ]

[ IG(Attendance)=1-left(frac48(0.811)+frac48(0.811)right) ]

[ boxed{IG(Attendance)=0.189} ]

Step 3: Information Gain for Assignments

Done:

- Pass = 3
- Fail = 1

Missed:

- Pass = 1
- Fail = 3

[ boxed{IG(Assignments)=0.189} ]

Step 4: Information Gain for Internal Marks

Good:

- Pass = 3
- Fail = 1

Poor:

- Pass = 1
- Fail = 3

[ boxed{IG(InternalMarks)=0.189} ]

All three attributes have equal information gain.

For a tie, choose Attendance as the root attribute using a fixed attribute-order rule.

Step 5: Final Decision Tree

Attendance?

High

Assignments?

Done → Pass

Missed → Internal Marks?

Good: Pass

Poor: Fail

Low

Assignments?

Done → Internal Marks?

Good: Pass

Poor: Fail

Missed → Fail

**Task 2.5 – Implement and Verify (2 Marks)**

Use the Iris dataset to test a decision-tree implementation from scratch and compare it with scikit-learn.

The following code shows a simple entropy-based decision tree using scikit-learn for verification. A separate from-scratch implementation is required for full assignment evidence.

```
from sklearn.datasets import load_irisfrom sklearn.model_selection import train_test_splitfrom sklearn.tree import DecisionTreeClassifierfrom sklearn.metrics import *X, y = load_iris(return_X_y=True)X1, X2, y1, y2 = train_test_split(    X, y, test_size=0.2, random_state=42)m = DecisionTreeClassifier(criterion="entropy", random_state=42)m.fit(X1, y1)p = m.predict(X2)print("Accuracy:", accuracy_score(y2, p))print("Precision:", precision_score(y2, p, average="macro"))print("Recall:", recall_score(y2, p, average="macro"))print("F1:", f1_score(y2, p, average="macro"))print(confusion_matrix(y2, p))
```

**Task 2.6 – Overfitting and Pruning (1 Mark)**

Overfitting occurs when a decision tree learns training examples too closely and performs poorly on new data.

Pruning reduces unnecessary branches.

| Model | Depth | Expected effect |
|---|---|---|
| Full tree | Higher | May overfit |
| Limited depth | Lower | Simpler model |
| Pruned tree | Controlled | Better generalisation |

A smaller tree may have slightly lower training accuracy but better validation performance.

**Task 2.7 – Challenge Your Design (1 Mark)**

Change one value in the dataset.

For example, change record 4:

Before: High, Missed, Poor → Fail.

After: High, Missed, Poor → Pass.

Prediction: The information gain values may change, and the root attribute may change.

After modification, recalculate the entropy and information gain to verify the root.

Identifier-like attribute:

Add StudentID with unique values such as S01, S02, S03.

Information Gain may favour this attribute because it creates pure individual groups.

However, it cannot predict new students.

Repair: Remove identifier attributes and use pruning or minimum-sample restrictions.

Result: Decision trees classify student performance using information gain and can be improved through pruning to reduce overfitting.

**TASK 3 – Pseudocode Forensics: Hidden Variables and the EM Algorithm (6 Marks)**

**Task 3.1 – Locate the Defect (1 Mark)**

The defect is in the E-step of the Gaussian Mixture Model.

Original statement:

r[i][k] = pi[k] * Normal(X[i] | mu[k], var[k])

The responsibility values are not normalised.

Responsibilities represent the probability that a data point belongs to a particular Gaussian component.

The sum of responsibilities for each data point must equal 1.

[ sum_{k=1}^{K}r_{ik}=1 ]

Without normalisation, the mixing weights may not sum to 1, and the estimated means and variances become incorrect.

**Task 3.2 – Construct a Failing Input and Trace (1 Mark)**

Consider the dataset:

[ X={0,1,5,6} ]

Number of Gaussian components:

[ K=2 ]

Initial parameters:

| Parameter | Gaussian 1 | Gaussian 2 |
|---|---|---|

| | | |
|---|---|---|
| Mixing weight | 0.5 | 0.5 |
| Mean | 0 | 6 |
| Variance | 1 | 1 |

For data point (X=0):

[ r_1=0.5times Normal(0|0,1) ]

[ r_1=0.1995 ]

Similarly:

[ r_2=0.5times Normal(0|6,1) ]

[ r_2approx0 ]

The total responsibility is approximately 0.1995 instead of 1.

This is the first incorrect value.

**Task 3.3 – Repair the Algorithm (1 Mark)**

Replace the original responsibility calculation with the following:

FOR each data point i:

    Calculate responsibility for every component

    total = SUM of all responsibilities

    FOR each component k:

        r[i][k] = r[i][k] / total

END FOR

The corrected formula is:

[ r_{ik}= frac{pi_kN(x_i|mu_k,sigma_k^2)} {sum_{j=1}^{K}pi_jN(x_i|mu_j,sigma_j^2)} ]

Now the responsibilities of each data point sum to 1.

**Task 3.4 – Implement Faulty and Repaired Versions (2 Marks)**

Use Python to implement both versions.

import numpy as npfrom sklearn.mixture import GaussianMixtureX = np.array([0, 1, 5, 6]).reshape(-1, 1)for seed in range(20):  model = GaussianMixture( n_components=2,  random_state=seed,  max_iter=100  )  model.fit(X)  print("Seed:", seed)  print("Weights:", model.weights_)  print("Log likelihood:", model.lower_bound_)

This is a reference implementation using a library, not the required from-scratch implementation.

For the full experiment, implement the E-step and M-step yourself and compare:

| **Measure** | **Faulty EM** | **Repaired EM** |
|---|---|---|
| Responsibilities | Not normalised | Normalised |
| Mixing weights | May not sum to 1 | Sum to 1 |
| Log-likelihood | May decrease | Non-decreasing under exact EM updates |
| Parameter estimates | Incorrect | Correctly estimated |

Run at least 20 initialisations and plot the log-likelihood for each iteration.

**Task 3.5 – Regression Check (1 Mark)**

A regression check verifies that the corrected algorithm still works on inputs where the original algorithm was already correct.

For example, use a dataset containing two well-separated groups:

[ X={0,0.1,5,5.1} ]

Use two Gaussian components with equal initial weights.

The corrected algorithm should identify two groups around 0 and 5.

Result: Normalising the responsibilities repairs the EM algorithm and ensures valid probability estimates.

**TASK 4 – Neural Network Back-Propagation (9 Marks)**

**Task 4.1 – Define the State (1 Mark)**

A neural network contains input, hidden and output layers.

The important quantities are:

1. Weighted input

[ z^{(l)}=W^{(l)}a^{(l-1)}+b^{(l)} ]

2. Activation

[ a^{(l)}=f(z^{(l)}) ]

3. Error term

[ delta^{(l)}=frac{partial E}{partial z^{(l)}} ]

Where:

- W = Weight matrix.
- a = Activation vector.
- b = Bias vector.
- f = Activation function.
- E = Error or loss.

For layer (l), if it has (n_l) neurons:

- (z^{(l)}): (n_ltimes1)
- (a^{(l)}): (n_ltimes1)
- (delta^{(l)}): (n_ltimes1)
- (W^{(l)}): (n_ltimes n_{l-1})

**Task 4.2 – Forward and Backward Recurrences (2 Marks)**

Forward propagation

Calculate the weighted sum:

[ z^{(l)}=W^{(l)}a^{(l-1)}+b^{(l)} ]

Calculate activation:

[ a^{(l)}=sigma(z^{(l)}) ]

Sigmoid function:

[ sigma(z)=frac{1}{1+e^{-z}} ]

Backward propagation

At the output layer:

[ delta^{(L)}=(a^{(L)}-y)odotsigma'(z^{(L)}) ]

For hidden layers:

[ delta^{(l)}= (W^{(l+1)})^Tdelta^{(l+1)} odotsigma'(z^{(l)}) ]

Weight gradient:

[ frac{partial E}{partial W^{(l)}}= delta^{(l)}(a^{(l-1)})^T ]

Bias gradient:

[ frac{partial E}{partial b^{(l)}}=delta^{(l)} ]

Weights are updated using:

[ W=W-etafrac{partial E}{partial W} ]

Where (eta) is the learning rate.

Why backward from last layer to first?

The output error is calculated first. Hidden-layer errors depend on the errors of the next layer. Therefore, calculations move backwards through the network.

**Task 4.3 – Complete Pseudocode (2 Marks)**

```
ALGORITHM NeuralNetworkTraining

BEGIN

Initialize weights and biases randomly

REPEAT:

    Divide training data into mini-batches

    FOR each mini-batch:

        FOR each input:
            Perform forward propagation
            Calculate predicted output
            Calculate loss

        Calculate output layer error

        FOR hidden layers from last to first:
```

Calculate error using chain rule

Calculate weight gradients

Calculate bias gradients

Update weights

Update biases

Calculate average training loss

UNTIL maximum epochs reached

OR loss change is below threshold

RETURN trained network

END

**Task 4.4 – Desk-Check (1 Mark)**

Given:

- Input: (x=(1,0))
- Target: (y=1)
- Learning rate: 0.5
- Hidden weights: (0.5, -0.4), (0.3, 0.8)
- Output weights: (0.6, -0.2)
- All biases = 0

Step 1: Hidden layer calculation

First hidden neuron:

[ z_1=0.5(1)-0.4(0)=0.5 ]

[ a_1=sigma(0.5)=0.6225 ]

Second hidden neuron:

[ z_2=0.3(1)+0.8(0)=0.3 ]

[ a_2=sigma(0.3)=0.5744 ]

Step 2: Output layer calculation

[ z_3=0.6(0.6225)-0.2(0.5744) ]

[ z_3=0.2586 ]

[ y_{pred}=sigma(0.2586) ]

[ boxed{y_{pred}=0.5643} ]

Step 3: Calculate squared error

Using half squared-error loss:

[ E=frac12(y-y_{pred})^2 ]

[ E=frac12(1-0.5643)^2 ]

[ boxed{E=0.0949} ]

Step 4: Output error

[ delta_3=(0.5643-1)(0.5643)(1-0.5643) ]

[ delta_3=-0.1071 ]

Step 5: Hidden errors

[ delta_1=-0.1071(0.6)(0.6225)(1-0.6225) ]

[ delta_1=-0.0151 ]

[ delta_2=-0.1071(-0.2)(0.5744)(1-0.5744) ]

[ delta_2=0.00524 ]

Step 6: Update output weights

[ w_1=0.6-0.5(-0.1071times0.6225) ]

[ boxed{w_1=0.6333} ]

[ w_2=-0.2-0.5(-0.1071times0.5744) ]

[ boxed{w_2=-0.1692} ]

Step 7: Update hidden weights

Since the second input is zero, its incoming weights remain unchanged in this update.

[ w_{11}=0.5-0.5(-0.0151)=0.5076 ]

[ w_{21}=0.3-0.5(0.00524)=0.2974 ]

Updated weights:

| Weight | Before | After |
|---|---|---|
| Hidden 1, input 1 | 0.5000 | 0.5076 |
| Hidden 1, input 2 | -0.4000 | -0.4000 |
| Hidden 2, input 1 | 0.3000 | 0.2974 |
| Hidden 2, input 2 | 0.8000 | 0.8000 |
| Output 1 | 0.6000 | 0.6333 |
| Output 2 | -0.2000 | -0.1692 |

Result: Back-propagation calculates errors from the output layer to the hidden layers and updates weights to reduce prediction error.

**Task 4.5 – Implement and Verify (2 Marks)**

A simple NumPy implementation for XOR:

import numpy as npX = np.array([[0,0],[0,1],[1,0],[1,1]])Y = np.array([[0],[1],[1],[0]])def sig(x):  return 1/(1+np.exp(-x))np.random.seed(1)W1 = np.random.randn(2,4)W2 = np.random.randn(4,1)lr = 0.5for epoch in range(10000):  H = sig(X @ W1)  O = sig(H @ W2)  dO = (O-Y)*O*(1-O)  dW2 = H.T @ dO  dH = (dO @ W2.T)*H*(1-H)  dW1 = X.T @ dH  W2 -= lr*dW2  W1 -= lr*dW1print(np.round(O))

Expected output after successful training:

[[0]

 [1]

 [1]

 [0]]

Gradient verification:

Numerical gradients can be calculated using small changes to each weight.

[ g_{num}=frac{E(w+epsilon)-E(w-epsilon)}{2epsilon} ]

Compare numerical and analytical gradients.

A small maximum relative error indicates that back-propagation is implemented correctly.

**Task 4.6 – Stress the Design (1 Mark)**

Use the 8 × 8 digits dataset.

Test hidden units:

[ {2,8,32,64} ]

Learning rates:

[ {0.001,0.01,0.1,1.0} ]

Activations:

- Sigmoid
- ReLU

Expected observations:

| Condition | Expected behaviour |
|---|---|
| 2 hidden units | May underfit |
| 8 hidden units | Moderate learning |
| 32 hidden units | Higher learning capacity |
| 64 hidden units | More computation |
| Very high learning rate | May diverge |
| Very low learning rate | Slow convergence |

Use training and validation loss curves, accuracy, macro-F1 and confusion matrix.

A vanishing-gradient problem can occur when sigmoid activations become saturated. The gradient becomes very small, and earlier layers learn slowly.

Result: Neural networks learn through forward propagation and back-propagation. The number of hidden neurons, learning rate and activation function affect performance.

**TASK 5 – Same Dataset, Different Learning Reasoning (8 Marks)**

**Task 5.1 – Statistical Learning with Complete Data: Naive Bayes (3 Marks)**

**A. Design – Naive Bayes Algorithm (1 Mark)**

Naive Bayes is a classification algorithm based on Bayes' theorem.

It assumes that input features are conditionally independent given the class.

Bayes' theorem:

[ P(C|X)=frac{P(X|C)P(C)}{P(X)} ]

Where:

- C = Class.
- X = Input features.
- P(C) = Prior probability.
- P(X|C) = Likelihood.
- P(C|X) = Posterior probability.

Pseudocode

BEGIN

Read training dataset

Calculate probability of each class

FOR each feature:

Calculate feature probability

Apply Laplace smoothing

FOR each new input:

Calculate probability for every class

Select class with highest probability

Display predicted class

END

Laplace smoothing formula:

[ P(x|C)=frac{Count(x,C)+alpha} {Count(C)+alpha V} ]

Where:

- (alpha) = Smoothing parameter.

- V = Number of possible feature values.

Zero-count example

Consider:

| Record | Weather | Overload |
|---|---|---|
| 1 | Hot | Yes |
| 2 | Cold | No |
| 3 | Cold | No |

Suppose a new record has Weather = Hot.

Without smoothing, the probability of Cold given Overload = Yes is zero because no such example exists.

Multiplying probabilities may produce zero.

With Laplace smoothing, zero probabilities are avoided.

**B. Implementation (2 Marks)**

Test different smoothing values:

[ alpha={0,0.1,1,10} ]

from sklearn.naive_bayes import GaussianNBfrom sklearn.datasets import load_irisfrom sklearn.model_selection import train_test_splitfrom sklearn.metrics import *X, y = load_iris(return_X_y=True)a, b, c, d = train_test_split(  X, y, test_size=0.2, random_state=42)m = GaussianNB()m.fit(a, c)p = m.predict(b)print("Accuracy:", accuracy_score(d, p))print("Precision:", precision_score(d, p, average="macro"))print("Recall:", recall_score(d, p, average="macro"))print("F1:", f1_score(d, p, average="macro"))print("Log Loss:", log_loss(d, m.predict_proba(b)))

Note: GaussianNB uses variance smoothing rather than the categorical Laplace formula. For the required experiment, implement categorical or multinomial Naive Bayes from scratch to compare the four specified alpha values.

**Task 5.2 – Kernel Machines (3 Marks)**

**A. Design (1 Mark)**

When hazardous and normal conditions are not linearly separable, a linear decision boundary becomes insufficient.

A kernel machine solves this problem by transforming data into a higher-dimensional feature space.

Important concepts:

1. Margin: Distance between the decision boundary and the nearest training points.
2. Support vectors: Data points closest to the decision boundary.
3. Kernel function: Measures similarity between data points and helps create nonlinear decision boundaries.

Common kernels:

- Linear kernel.
- Polynomial kernel.
- RBF kernel.

RBF kernel formula:

[ K(x,y)=e^{-gamma||x-y||^2} ]

Where gamma controls the influence of individual training points.

**B. Implementation (2 Marks)**

Compare Linear SVM and RBF SVM.

from sklearn.datasets import load_irisfrom sklearn.model_selection import train_test_split, GridSearchCVfrom sklearn.svm import SVCfrom sklearn.metrics import accuracy_score, f1_scoreimport timeX, y = load_iris(return_X_y=True)a, b, c, d = train_test_split( X, y, test_size=0.2, random_state=42)for kernel in ["linear", "rbf"]: m = SVC(kernel=kernel) params = {"C": [0.1, 1, 10]} if kernel == "rbf": params["gamma"] = [0.01, 0.1, 1] g = GridSearchCV(m, params, cv=5) start = time.time() g.fit(a, c) p = g.predict(b) print(kernel) print("Accuracy:", accuracy_score(d, p)) print("F1:", f1_score(d, p, average="macro")) print("Support vectors:", g.best_estimator_.n_support_) print("Time:", time.time()-start)

Compare the following:

| **Model** | **Accuracy** | **F1-score** | **Support vectors** | **Training time** |
|---|---|---|---|---|
| Naive Bayes | Measure | Measure | N/A | Measure |
| Decision Tree | Measure | Measure | N/A | Measure |
| Linear SVM | Measure | Measure | Measure | Measure |
| RBF SVM | Measure | Measure | Measure | Measure |

Use the same dataset split for all models. For the smart-grid requirement, a labelled overload dataset should replace Iris in the final experiment.

**Task 5.3 – Explanation-Based Learning (2 Marks)**

**A. Design – Domain Theory (1 Mark)**

Explanation-Based Learning (EBL) learns a general rule by explaining one example using existing domain knowledge.

Consider the following six rules:

1. If current exceeds the safe limit, current is high.
2. If voltage is below the safe limit, voltage is low.
3. If temperature exceeds the safe limit, temperature is high.
4. If current is high, there is an electrical overload condition.
5. If temperature is high and current is high, equipment overheating risk exists.
6. If an electrical overload condition exists and equipment overheating risk exists, the substation is hazardous.

Example:




Current = 150 A, safe current limit = 100 A.

Temperature = 90°C, safe temperature limit = 70°C.

Explanation tree:

Substation is Hazardous

AND

Current is High

150 A exceeds 100 A

Temperature is High

90°C exceeds 70°C

Generalised operational rule:

IF current > safe_current_limit

AND temperature > safe_temperature_limit

THEN substation = HAZARDOUS

**B. Implementation (1 Mark)**

Test the learned rule on 20 cases.

Example:

| Case | Current | Temperature | Expected |
|---|---|---|---|
| 1 | High | High | Hazardous |
| 2 | High | Normal | Not hazardous |
| 3 | Normal | High | Not hazardous |
| 4 | Normal | Normal | Not hazardous |

Continue with 20 combinations or designed test cases.

Compare:

- Inference using the original domain theory.
- Direct application of the learned rule.

The learned rule reduces repeated reasoning because the explanation has already been generalised.

What was learned?

The system learned a reusable operational rule from one explained example. However, the rule is logically derived from the existing domain theory; it does not introduce new knowledge beyond that theory.

Result: Naive Bayes, kernel machines and explanation-based learning use different approaches to classify and reason about hazardous conditions.

**TASK 6 – Design Challenge: Reinforcement Learning for an Energy-Aware Smart Controller (13 Marks)**

**Stage 1 – Interpret the Problem (2 Marks)**

A university lecture hall uses a smart air-conditioning controller.

The controller must select one of three actions every 15 minutes.

Input:

- Room occupancy.
- Indoor temperature.
- Outdoor temperature.
- Electricity price.
- Current time.

Output:

- OFF.
- LOW.
- HIGH.

Objective:

Reduce electricity cost while maintaining a comfortable temperature whenever the room is occupied.

Constraints:

1. The room must remain comfortable during occupancy.
2. Energy usage should be reduced.
3. Electricity cost should be minimised.
4. The controller must make decisions every 15 minutes.

Episode beginning: At the start of a daily operating period.

Episode ending: At the end of the operating period.

**Stage 2 – Propose the Strategy (2 Marks)**

Different approaches can be used.

| **Algorithm** | **Description** |
|---|---|
| Fixed-rule thermostat | Uses fixed temperature limits |
| Supervised learning | Learns from previous operator decisions |
| Q-Learning | Learns the best action using rewards |
| SARSA | Learns using the next action actually selected |
| Model-based learning | Uses an environment model to predict future results |

Selected algorithm: Q-Learning

Reasons:

1. The controller makes repeated decisions.
2. Each action produces a reward.
3. The environment changes over time.
4. The controller can learn through trial and error.
5. It can improve decisions using previous experiences.

Q-Learning is suitable because it learns action values without requiring a complete model of the environment.

**Stage 3 – Identify the Critical Information (2 Marks)**

**1. State Representation**

The state is:

[ S=(O,T_i,T_o,P,H) ]

Where:

- O = Occupancy.
- (T_i) = Indoor temperature category.
- (T_o) = Outdoor temperature category.
- P = Electricity price category.
- H = Time period.

These variables summarise the information used to make the next decision. For a more accurate Markov state, include recent temperature trends or previous AC settings if they influence future temperature.

**2. Action Set**

[ A={OFF,LOW,HIGH} ]

**3. Reward Function**

[ R=-(EnergyCost+ComfortPenalty) ]

Where:

[ EnergyCost=EnergyUsedtimes ElectricityPrice ]

Comfort penalty:

[ ComfortPenalty= begin{cases} 0, & text{If comfortable} 10, & text{If occupied and uncomfortable} end{cases} ]

A larger negative reward means a worse action.

**4. Exploration Strategy**

Use epsilon-greedy exploration.

- With probability ε, select a random action.
- Otherwise, select the action with the highest Q-value.

**5. Discount Factor**

Select:

[ gamma=0.9 ]

This allows the controller to consider future rewards as well as immediate rewards.

**Stage 4 – Complete Pseudocode (3 Marks)**

Algorithm: EnergyAwareQLearning

BEGIN

Initialize Q-table with zeros

Set learning rate alpha = 0.1

Set discount factor gamma = 0.9

Set exploration rate epsilon = 0.1

FOR each episode:

    Initialize room environment

    WHILE episode is not finished:

        Observe current state

        Generate available actions

        With probability epsilon:

            Select random action

        Otherwise:

            Select action with maximum Q-value

        Apply selected action

        Observe next state

        Calculate energy cost

        Calculate comfort penalty

Calculate reward

Update Q-value:

Q(s,a) = Q(s,a) + alpha *

[reward + gamma * max Q(next state, actions)

- Q(s,a)]

Set current state = next state

END WHILE

END FOR

Extract policy by selecting the action

with maximum Q-value for each state

Display learned policy

END

**Stage 5 – Desk-Check (1 Mark)**

Consider a simple environment with three states:

- S1 = Room is comfortable.
- S2 = Room is warm.
- S3 = Room is very hot.

Actions:

- OFF.
- HIGH.

Initial Q-values:

[ Q(s,a)=0 ]

Parameters:

[ alpha=0.5,quadgamma=0.9 ]

Q-Learning formula:

[ Q(s,a)=Q(s,a)+alpha[r+gammamax Q(s',a')-Q(s,a)] ]

Four manual updates:

| Step | State | Action | Reward | Next state |
|---|---|---|---|---|

| | | | | |
|---|---|---|---|---|
| 1 | S1 | OFF | 2 | S1 |
| 2 | S2 | HIGH | 5 | S1 |
| 3 | S3 | HIGH | 8 | S2 |
| 4 | S2 | OFF | -4 | S3 |

Assume all Q-values are initially zero.

Update 1:

[ Q(S1,OFF)=0+0.5(2)=1 ]

Update 2:

[ Q(S2,HIGH)=0+0.5(5)=2.5 ]

Update 3:

[ Q(S3,HIGH)=0+0.5(8)=4 ]

Update 4:

The maximum Q-value for S3 is 4.

[ Q(S2,OFF)=0+0.5[-4+0.9(4)] ]

[ =0.5(-0.4) ]

[ boxed{Q(S2,OFF)=-0.2} ]

Final Q-table:

| State | OFF | HIGH |
|---|---|---|
| S1 | 1.0 | 0 |
| S2 | -0.2 | 2.5 |
| S3 | 0 | 4.0 |

The controller selects the action with the highest Q-value in each state.

**Stage 6 – Implement, Instrument and Attack Your Own Design (3 Marks)**

**A. Python Implementation**

import randomstates = 3actions = 3Q = [[0]*actions for _ in range(states)]alpha = 0.1gamma = 0.9epsilon = 0.1def reward(s, a):  return -1 if a == 0 else 2 if a == 1 else 3for episode in range(1000):  s = random.randint(0, 2)  for step in range(20):  if random.random() < epsilon:  a = random.randint(0, 2)  else:  a = Q[s].index(max(Q[s]))  r = reward(s, a)  ns = random.randint(0, 2)  Q[s][a]

+= alpha * ( r + gamma * max(Q[ns]) - Q[s][a] ) s = nsprint("Final Q Table")for row in Q: print([round(x, 2) for x in row])

This is a basic Q-learning demonstration. For the full assignment, replace the random transition and reward functions with a simulator that models room temperature, occupancy, electricity price and actual energy consumption.

**B. Performance Measures**

Record results for at least 20 random seeds.

| **Measure** | **Q-Learning** | **Fixed-rule baseline** |
|---|---|---|
| Average reward | Measure | Measure |
| Energy used (kWh) | Measure | Measure |
| Comfort violations | Measure | Measure |
| Episodes to convergence | Measure | N/A |
| Execution time | Measure | Measure |

Use a moving average of episode rewards to create a learning curve.

**C. Attack Your Own Design**

Consider an unusual situation:

Electricity prices become very high just before students enter the classroom.

The controller may learn to keep the AC OFF to save electricity.

However, when students arrive, the room may become uncomfortable.

Predicted behaviour:

- The controller may select OFF frequently.
- Energy cost may decrease.
- Comfort violations may increase.

Reason:

The reward function may give too much importance to electricity cost and too little importance to comfort.

**D. Improvement**

Increase the comfort penalty when the room is occupied.

Also include occupancy schedules and pre-cooling decisions in the state representation.

This helps the controller prepare the room before students arrive.

Result: Q-Learning can develop an energy-aware AC control policy by learning from rewards. A properly designed reward function is important to balance electricity savings and student comfort.

**Overall Conclusion**

Artificial Intelligence learning algorithms help systems learn from data, examples and interactions.

| Task | Topic | Main concept |
|---|---|---|
| 1 | Learning protocol | Proper training and evaluation |
| 2 | Decision trees | Information Gain and classification |
| 3 | EM algorithm | Learning with hidden variables |
| 4 | Neural networks | Back-propagation |
| 5 | Learning methods | Naive Bayes, SVM and EBL |
| 6 | Reinforcement learning | Q-Learning and reward-based decisions |

Final conclusion: This assignment demonstrates how learning algorithms are designed, implemented, tested and improved to solve real-world problems.